

Отчёт по лабораторной работе №3

Измерение и тестирование пропускной способности сети

Воспроизводимый эксперимент

Базлов В.А.

Содержание

Цель работы

Задание

Выполнение работы

3.1. Подготовка окружения и проекта

3.1.1. Подготовка рабочего каталога

3.1.2. Создание топологии через API Mininet

3.1.3. Проверка топологии

3.2. Вывод параметров хостов

3.2.1. Информация о хосте h1

3.2.2. Информация об обоих хостах

3.3. Ограничение производительности

3.3.1. CPULimitedHost и TCLink

3.3.2. Настройка h1 и h2

3.3.3. Настройка сетевого канала

3.4. Воспроизводимый эксперимент iPerf3

3.4.1. Подготовка lab_iperf3.py

3.4.2. Запуск сервера и клиента

3.4.3. Экспорт результатов в JSON

3.5. Построение графиков

3.5.1. Обработка JSON-файла

3.5.2. Результаты визуализации

3.6. Автоматизация с помощью Makefile

3.6.1. Создание Makefile

3.6.2. Проверка make clean и make

3.7. Завершение работы

Выводы

Цель работы

Основной целью работы является знакомство с инструментом для измерения пропускной способности сети в режиме реального времени — iPerf3, а также получение навыков проведения воспроизводимого эксперимента по измерению пропускной способности моделируемой сети в среде Mininet.

Задание

1. Воспроизвести посредством API Mininet эксперименты по измерению пропускной способности с помощью iPerf3.
2. Построить графики по проведённому эксперименту.

Выполнение работы

3.1. Подготовка окружения и проекта

3.1.1. Подготовка рабочего каталога

Для выполнения работы используется каталог `~/work/lab_iperf3`. В нём создан подкаталог `lab_iperf3_topo`, после чего файл `mininet/examples/emptynet.py` скопирован в рабочий каталог и переименован в `lab_iperf3_topo.py`.

```
mininet@mininet-vm:~$ mkdir -p ~/work/lab_iperf3
mininet@mininet-vm:~$ cd ~/work/lab_iperf3
mininet@mininet-vm:~/work/lab_iperf3$ pwd
/home/mininet/work/lab_iperf3
mininet@mininet-vm:~/work/lab_iperf3$ cp ~/mininet/examples/emptynet.py ~/work/
lab_iperf3/lab_iperf3_topo.py
mininet@mininet-vm:~/work/lab_iperf3$ cd ~/work/lab_iperf3
mininet@mininet-vm:~/work/lab_iperf3$ ls -l
total 24
drwxrwxr-x 2 mininet mininet 4096 Oct  7 09:07 iperf3
-rw-r--r-- 1 mininet mininet 7795 Sep 24 04:10 iperf3_results.json
-rw-rw-r-- 1 mininet mininet  951 Sep 24 04:13 iperf.csv
-rwxrwxr-x 1 mininet mininet  985 Oct  8 05:17 lab_iperf3_topo.py
drwxrwxr-x 2 mininet mininet 4096 Sep 24 04:13 results
mininet@mininet-vm:~/work/lab_iperf3$
```

Подготовка рабочего каталога и копирование `emptynet.py`

3.1.2. Создание топологии через API Mininet

Сценарий `lab_iperf3_topo.py` создаёт сеть с контроллером `c0`, двумя хостами `h1` и `h2` и коммутатором `s3`. Хостам назначаются IP-адреса `10.0.0.1` и `10.0.0.2`. Между каждым хостом и коммутатором создаётся двунаправленная связь.

Основные строки создания топологии

```
from mininet.cli import CLI
from mininet.log import setLogLevel, info

def emptyNet():

    "Create an empty network and add nodes to it."

    net = Mininet( controller=Controller, waitConnected=True )

    info( '*** Adding controller\n' )
    net.addController( 'c0' )

    info( '*** Adding hosts\n' )
    h1 = net.addHost( 'h1', ip='10.0.0.1' )
    h2 = net.addHost( 'h2', ip='10.0.0.2' )

    info( '*** Adding switch\n' )
    s3 = net.addSwitch( 's3' )

    info( '*** Creating links\n' )
    net.addLink( h1, s3 )
    net.addLink( h2, s3 )

    info( '*** Starting network\n' )
    net.start()
```

```
from mininet.cli import CLI
from mininet.log import setLogLevel, info

def emptyNet():

    "Create an empty network and add nodes to it."

    net = Mininet( controller=Controller, waitConnected=True )

    info( '*** Adding controller\n' )
    net.addController( 'c0' )

    info( '*** Adding hosts\n' )
    h1 = net.addHost( 'h1', ip='10.0.0.1' )
    h2 = net.addHost( 'h2', ip='10.0.0.2' )

    info( '*** Adding switch\n' )
    s3 = net.addSwitch( 's3' )

    info( '*** Creating links\n' )
    net.addLink( h1, s3 )
    net.addLink( h2, s3 )

    info( '*** Starting network\n' )
    net.start()
```

Листинг lab_iperf3_topo.py

3.1.3. Проверка топологии

После запуска скрипта выполнена проверка элементов топологии средствами CLI Mininet с помощью команд `net`, `links` и `dump`. После проверки работа Mininet завершена командой `exit`.

```

mininet@mininet-vm:~/work/lab_iperf3$ cd ~/work/lab_iperf3
mininet@mininet-vm:~/work/lab_iperf3$ sudo python lab_iperf3_topo.py
*** Adding controller
*** Adding hosts
*** Adding switch
*** Creating links
*** Starting network
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s3 ...
*** Waiting for switches to connect
s3
*** Running CLI
*** Starting CLI:
mininet> net
h1 h1-eth0:s3-eth1
h2 h2-eth0:s3-eth2
s3 lo: s3-eth1:h1-eth0 s3-eth2:h2-eth0
c0
mininet> links
h1-eth0<->s3-eth1 (OK OK)
h2-eth0<->s3-eth2 (OK OK)
mininet> dump
<Host h1: h1-eth0:10.0.0.1 pid=3406>
<Host h2: h2-eth0:10.0.0.2 pid=3408>
<OVSSwitch s3: lo:127.0.0.1,s3-eth1:None,s3-eth2:None pid=3413>

```

Результат выполнения net, links и dump

3.2. Вывод параметров хостов

3.2.1. Информация о хосте h1

После запуска сети в сценарий добавлена команда вывода имени, IP-адреса и MAC-адреса хоста h1. Для этого используются h1.name, h1.IP() и h1.MAC().

3.2.2. Информация об обоих хостах

Затем аналогичный вывод реализован для h1 и h2. Корректность изменения проверяется повторным запуском сценария.

```

mininet> h1 ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
2: h1-eth0@if12: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether aa:4f:3f:9c:b4:e2 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.0.0.1/8 brd 10.255.255.255 scope global h1-eth0
        valid_lft forever preferred_lft forever
mininet> h1 ip link
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: h1-eth0@if12: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP mode DEFAULT group default qlen 1000
    link/ether aa:4f:3f:9c:b4:e2 brd ff:ff:ff:ff:ff:ff link-netnsid 0
mininet>

```

Вывод параметров обоих хостов

3.3. Ограничение производительности

3.3.1. CPULimitedHost и TCLink

Создана копия сценария lab_iperf3_topo2.py. В начало файла добавлены импорты CPULimitedHost и TCLink. Объект Mininet настроен на использование CPULimitedHost в качестве класса хоста и TCLink в качестве класса канала.

```

mininet> net
h1 h1-eth0:s3-eth1
h2 h2-eth0:s3-eth2
s3 lo: s3-eth1:h1-eth0 s3-eth2:h2-eth0
c0
mininet> links
h1-eth0<->s3-eth1 (OK OK)
h2-eth0<->s3-eth2 (OK OK)
mininet> dump
<CPULimitedHost h1: h1-eth0:10.0.0.1 pid=3969>
<CPULimitedHost h2: h2-eth0:10.0.0.2 pid=3971>
<OVSSwitch s3: lo:127.0.0.1,s3-eth1:None,s3-eth2:None pid=3976>
<Controller c0: 127.0.0.1:6653 pid=3962>
mininet>

```

Листинг с импортом CPULimitedHost и TCLink

3.3.2. Настройка h1 и h2

Для h1 задано выделение 50% общих ресурсов процессора, для h2 — 45%.

```
mininet@mininet-vm:~/work/lab_iperf3$ cd ~/work/lab_iperf3
mininet@mininet-vm:~/work/lab_iperf3$ grep -n "addHost" lab_iperf3_topo2.py
18:     h1 = net.addHost('h1', cpu=0.5)
19:     h2 = net.addHost('h2', cpu=0.45)
mininet@mininet-vm:~/work/lab_iperf3$
```

Настройки CPU для h1 и h2

3.3.3. Настройка сетевого канала

Соединение h1 с коммутатором настроено с пропускной способностью 10 Мбит/с, задержкой 5 мс, максимальным размером очереди 1000 пакетов и потерями 10%. Используется ограничитель HTB.

```
mininet@mininet-vm:~/work/lab_iperf3$ grep -n -E "bw=10|delay='5ms'|max_queue_s
ize=1000|loss=10|use_htb=True" lab_iperf3_topo2.py
28:         bw=10,
29:         delay='5ms',
30:         max_queue_size=1000,
31:         loss=10,
32:         use_htb=True
mininet@mininet-vm:~/work/lab_iperf3$
```

Настройка TCLink и параметры канала

3.4. Воспроизводимый эксперимент iPerf3

3.4.1. Подготовка lab_iperf3.py

На основе lab_iperf3_topo2.py создан сценарий lab_iperf3.py. В него добавлен модуль time. Согласно заданию, в итоговом сценарии на хостах отсутствует ограничение использования ресурсов CPU, а каналы настроены на 100 Мбит/с с задержкой 75 мс, без потерь, без ограничителя пропускной способности и без ограничения максимального размера очереди.

```
mininet@mininet-vm:~/work/lab_iperf3$ grep -n -E "import time|addHost|addLink|C
PULimitedHost|TCLink|bw=|delay=|loss=|max_queue" lab_iperf3.py
5:import time
15:     h1 = net.addHost('h1', ip='10.0.0.1')
16:     h2 = net.addHost('h2', ip='10.0.0.2')
22:     net.addLink(h1, s3, bw=100, delay='75ms')
23:     net.addLink(h2, s3, bw=100, delay='75ms')
mininet@mininet-vm:~/work/lab_iperf3$
```

Листинг lab_iperf3.py с настройками топологии

3.4.2. Запуск сервера и клиента

После запуска сети на хосте h2 запускается сервер iPerf3. Через 10 секунд на h1 запускается клиент iPerf3. CLI-интерфейс Mininet в данном сценарии закомментирован.

```

mininet@mininet-vm:~/work/lab_iperf3$ sudo python lab_iperf3.py
*** Adding controller
*** Add hosts
*** Add switches
*** Add links
*** Starting network
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s3 ...
*** Traffic generation
*** h2 : ('iperf3 -s -D -1',)
*** h1 : ('iperf3 -c', '10.0.0.2', '-J > iperf_result.json')
*** Stopping 1 controllers
c0
*** Stopping 2 links
..
*** Stopping 1 switches
s3
*** Stopping 2 hosts
h1 h2
*** Done
mininet@mininet-vm:~/work/lab_iperf3$

```

Команды запуска сервера и клиента iPerf3

3.4.3. Экспорт результатов в JSON

Ключ -s запускает iPerf3 в режиме сервера, -D переводит сервер в фоновый режим, а -1 ограничивает сервер одним клиентом. В клиентском вызове ключ -с задаёт адрес сервера, ключ -J включает JSON-формат вывода, а перенаправление > сохраняет результат в iperf_result.json.

```

mininet@mininet-vm:~/work/lab_iperf3$ echo $?
0
mininet@mininet-vm:~/work/lab_iperf3$

```

Результат запуска sudo python lab_iperf3.py

```

mininet@mininet-vm:~/work/lab_iperf3$ ls -lh iperf_result.json
-rw-r--r-- 1 root root 7.7K Oct  8 05:55 iperf_result.json
mininet@mininet-vm:~/work/lab_iperf3$ head -n 20 iperf_result.json
{
    "start": {
        "connected": [{
            "socket": 5,
            "local_host": "10.0.0.1",
            "local_port": 45608,
            "remote_host": "10.0.0.2",
            "remote_port": 5201
        }],
        "version": "iperf 3.7",
        "system_info": "Linux mininet-vm 5.4.0-42-generic #46-Ubuntu SMP Fri Jul 10 00:24:02 UTC 2020 x86_64",
        "timestamp": {
            "time": "Thu, 08 Oct 2026 12:55:18 GMT",
            "timesecs": 1791464118
        },
        "connecting_to": {
            "host": "10.0.0.2",
            "port": 5201
        },
        "cookie": "dsgfbidbxn6yg333kabga6yf4t5wpefekani",
    }
}
mininet@mininet-vm:~/work/lab_iperf3$

```

Проверка созданного iperf_result.json

3.5. Построение графиков

3.5.1. Обработка JSON-файла

После завершения эксперимента графики строятся из полученного JSON-файла командой `plot_iperf.sh iperf_result.json`.

```

mininet@mininet-vm:~/work/lab_iperf3$ ls -lh iperf_result.json
-rw-r--r-- 1 root root 7.7K Oct  8 05:55 iperf_result.json
mininet@mininet-vm:~/work/lab_iperf3$ plot_iperf.sh iperf_result.json
mininet@mininet-vm:~/work/lab_iperf3$

```

Запуск построения графиков

3.6. Завершение работы

После выполнения всех экспериментов соединение с виртуальной машиной Mininet завершается, виртуальная машина выключается.

Результаты выполнения работы

В ходе работы была создана программная топология Mininet из двух хостов и коммутатора, выполнена проверка её структуры и реализован вывод параметров хостов. Далее были изучены возможности ограничения производительности с помощью CPULimitedHost и TCLink.

Создан сценарий воспроизводимого эксперимента с iPerf3: сервер запускается на h2, клиент — на h1 с задержкой 10 секунд, а результаты сохраняются в формате JSON. На основании JSON-файла предусмотрено построение графиков.

Для автоматизации эксперимента создан Makefile, включающий запуск эксперимента, построение графиков и очистку каталога от результатов.

Выводы

В ходе лабораторной работы освоено использование API Mininet для программного создания и запуска сетевой топологии. Получены навыки работы с объектами Host, Switch и Link, а также с методами addHost(), addSwitch(), addLink(), start() и stop().

Изучено применение CPULimitedHost и TCLink для задания ограничений производительности виртуальных хостов и параметров сетевого соединения. Реализован воспроизводимый эксперимент с iPerf3, включающий автоматический запуск сервера и клиента и экспорт результатов в JSON.

Освоена обработка результатов эксперимента с помощью скрипта plot_iperf.sh и автоматизация последовательности действий посредством Makefile. Таким образом, выполнены основные требования лабораторной работы по воспроизведению эксперимента измерения пропускной способности средствами Mininet и iPerf3.